

字符串简单题选讲 2

Purslane

云斗学院 · 省选培训 · 第一阶段

2025.12.23

n 次方串

如果一个字符串可以写成 n 个 A 拼接而成，称之为一个 n 次方串。

记 $|A| = L$ ，如果 $|A|$ 的最小正周期 $p \mid L$ ，那么 $|A|$ 是 d 次方串成立，等价于 $d \mid \frac{L}{p}$ 。

否则， $|A|$ 最多只能是 1 次方串。

Runs 是处理 n 次方串的有利武器，但由于 Runs 实际上并不 practical，因此只提供[参考资料](#)。

QOJ15502 字符串问题

题目大意

给定系数序列 f ，定义 $w(S)$ 为 $f \frac{|S|}{d(S)}$ ，其中 $d(S)$ 为 S 的最小整周期。

给定字符串 S ，对每个 $1 \leq i \leq |S|$ ，求 $\sum_{j=1}^i w(S[j \cdots i])$ 。
 $1 \leq |S| \leq 10^6$ ，5s。

QOJ15502 字符串问题

记 $T(S) = |S|/d(S)$ 为字符串 S 的周期数。容易发现,
 $\forall t \mid T(S)$, T 是一个 t 次方串。

我们希望枚举 $S[j \cdots i]$ 的周期数。先对 f 使用莫比乌斯反演求出 g 使得: $\sum_{d|i} g_d = f_i$ 。

这样枚举周期 t , 把 g_t 加到答案里就行。 g_1 是平凡的, 考虑 $g_{\geq 2}$ 。进一步, 我们转化为枚举串的长度, 这样把 g 的一段前缀和加到答案里。

经典技巧: 优秀的拆分。如果希望串长为 t , 先按照 t 进行序列分块。容易发现, 每个块内往前延续的最长只有 $O(1)$ 种可能, 因此可以直接前缀和。

而如何计算往前延续的最长段? 发现只需要计算 $O(n \log n)$ 次 LCP, 使用二分 + 哈希可以做到 $O(n \log^2 n)$, 使用 SA 可以做到 $O(n \log n)$ 。

Z 函数

给定字符串 S ，定义 S 的 Z 函数 $z_i = \text{lcp}(S, S[i \cdot |S|])$ ，显然二分 + 哈希可以 $O(n \log n)$ 求出所有 Z 函数。

假设我们成功计算了 $z_{2,3,\dots,k}$ ， $\max_{2 \leq i \leq k} i + z_i - 1 = p$ ，并且在 $i = i_0$ 时取到。

那么， $S[k+1 \cdots p] = S[k-i_0+2 \cdots p-i_0+2]$ 。如果 $z_{k-i_0+2} < p-k$ 那么有 $z_{k+1} = z_{k-i_0+2}$ 。

否则，我们在 $p-k$ 的基础上暴力扩展，每扩展一次 $\max i + z_i - 1$ 都会增加 1，总复杂度 $O(n)$ 。

Z 函数

问题 1: 给定字符串 S 和 T , 如何对 S 的每个后缀求出和 T 的 LCP ?

问题 2: 如何使用 ExKMP 算法 (Z 函数) 来平替 KMP 算法 ?

Z 函数

问题 1: 给定字符串 S 和 T , 如何对 S 的每个后缀求出和 T 的 LCP?

问题 2: 如何使用 ExKMP 算法 (Z 函数) 来平替 KMP 算法?

解 1: 只需要把 T 和 S 拼接起来, 再跑 ExKMP 即可 (中间可以填上特殊字符);

Z 函数

问题 1: 给定字符串 S 和 T , 如何对 S 的每个后缀求出和 T 的 LCP?

问题 2: 如何使用 ExKMP 算法 (Z 函数) 来平替 KMP 算法?

解 1: 只需要把 T 和 S 拼接起来, 再跑 ExKMP 即可 (中间可以填上特殊字符);

解 2: 对每个 p , 只需要求出最小的 i , 使得 $i + z_i - 1 \geq p$, 则 $p - i + 1$ 就是 Border。将所有位置按照 $i + z_i - 1$ 排序 (注意由于值域是 $O(n)$ 的, 可以桶排)。

维护集合 Min 即可。

P12923 [POI 2021/2022 R3] 模板 2 / Szablon 2

题目大意

给定一个长度为 n 的字符串 S 。

如果购买一个字符串 T 的模具，则可以免费获得其反串的模具。
问有多少种可能得长度 L ，使得可以购买一个长度为 L 的模具，
能印出字符串 S （同一个位置不能印上不同的字符）。

$n \leq 10^6$, 4s。

P12923 [POI 2021/2022 R3] 模板 2 / Szablon 2

记 a_i 表示, $S[i \cdots n]$ 和 S 的 LCP。

b_i 表示, $S[1 \cdots n]$ 和 S 反串的最长公共后缀。

i 能被长度为 t 的模板覆盖, 有 $a_i \geq t$ 或者 $b_{i+t-1} \geq t$ 。

初始令 $p = 1$, 不断尝试找到最大的 $p \leq i \leq p + l$ 使得 $a_i \geq t$ 或者 $b_{i+t-1} \geq t$, 然后 $p \leftarrow i$, 直到 $p = n - l + 1$ 。

容易发现, 我们会选出若干段, 选出的第奇数个段都互补相邻——一共只会弄 $O(n \log n)$ 次。

而如何找 i ? 我们考虑以下问题: 维护集合 S , 在 S 中删数, 或找到 $\leq v$ 的最大数。容易使用并查集做到 $O(n \log n \alpha(n))$ 。

P10716 【MX-X1-T4】「KDOI-05」简单的字符串问题

题目大意

给定字符串 S 。 q 次询问，给定 k 和 i ，求有多少个字符串 A ，使得存在 $B_1, B_2, \dots, B_{k-1}$ 满足 $S[1 \cdots i] = AB_1AB_2 \cdots AB_{k-1}A$ (B 可以是空串)。

$n, q \leq 2 \times 10^5$, 2s。

P10716 【MX-X1-T4】「KDOI-05」简单的字符串问题

考虑 $|A| = a$ 合法，当且仅当：

- (1) $S[1 \cdots i]$ 有一个长度为 a 的 Border；
- (2) 从前往后考虑所有 $S[1 \cdots a]$ 出现的位置，选出 k 个无交的，右端点 $\leq i$ 。

注意到（注意不到也可以，会变成 $O(q \log n + n \log^2 n)$ ）合法的 a 应该对应 Border 树上 i 的一段祖先到根的链，因此只需要 $O(\log n)$ 次倍增判断。

现在相当于：如何 $O(1)$ 求出第二个问题。对每个 a 预处理 k ，发现 $ak \leq n$ ，所以 (a, k) 只有 $O(n \log n)$ 对。

预处理的过程相当于：给定位置 p 和数 v ，在序列 a 上求出最小的 $i \geq p$ 使得 $a_i \geq v$ （这个 a 就是 Z 函数）。

考虑从小到大枚举，问题变为：给定集合 S ，动态删数，求 $\geq v$ 的最小数。这个使用并查集可以做到 $O(n \log n \alpha(n))$ （我不太会分析并查集复杂度，这个应该是上限）。

P9576 「TAOI-2」 Ciallo

题目大意

给定字符串 S 和 T 。

定义 $f(S)$ 为, T 在 S 中出现次数。

对于每个 $1 \leq l \leq r \leq |S|$, 求出 S 删掉子串 $S[l \cdots r]$ 后 f 的和。

$|S| \leq 4 \times 10^5$, $|T| \leq 2 \times 10^5$, 2s。

P9576 「TAOI-2」 Ciallo

按照 T 在原始 S 中出现的位置分类。

如果 T 在 S 中出现的位置，没有被 $[l, r]$ 隔开，那么是比较显然的，求出 T 在 S 中所有出现位置即可。

否则，考虑 T 的头对应 S 中位置 i ， T 的尾对应 S 中位置 j 。

要求： $j - i + 1 > |T|$ ，此时方案为 $\max\{0, a_i + b_j - |T|\}$ （这里的 a 表示后缀和 T 的 LCP， b 表示前缀和 T 的 LCS）。

而这是容易使用树状数组 $O(n \log n)$ 维护的。

Manacher 算法

Manacher 算法主要研究回文串相关问题。研究回文串，很自然的想到去处理“回文中心”。

然而，回文中心有两种：一种恰好在某一个字符上，另一种是在两个字符中间。为了统一他们，我们在两个字符中间插入特殊字符，这样把所有的回文串长度统一为奇数。

枚举字符串中心，研究以这个字符为中心的最长回文串长度，容易发现这也是一个向外扩展的结构，可以模仿 ExKMP 算法、利用之前的信息，复杂度 $O(n)$ 。

Manacher 告诉我们一个深刻的事实：一个字符串 S 本质不同的回文子串只有 $O(n)$ 个，并且我们很容易求出的出现次数和位置。

P4287 [SHOI2011] 双倍回文

题目大意

如果字符串 S 是长度为 4 的倍数的回文串，且其前一半也是回文串，那么称 S 是双倍回文的。

给定字符串 S ，求 S 的最长双倍回文子串。

$|S| \leq 5 \times 10^5$, 1s。

P4287 [SHOI2011] 双倍回文

注意到我们很容易 $O(1)$ 去判断一个串是否是双倍回文的。
而我们提到，回文串本质上只有 $O(n)$ 个。在 Manacher 扩展到本质不同的串时 check 一下即可。
复杂度 $O(n)$ 。

[ABC349G] Palindrome Construction

题目大意

给定 n 和序列 $d_{1,2,\dots,n}$ 。请构造字符串 S (字符集为正整数), 使得以位置 i 为中心的最长回文串长度为 $2d_i + 1$ 。如果无解输出 -1。

$n \leq 2 \times 10^5$, $2s$ 。

[ABC349G] Palindrome Construction

一种带 $\log$ 的方法是：弄 $2n$ 个位置，让第 i 个位置和第 $2n + 1 - i$ 个相同。 $[l_i, r_i]$ 是回文串可以等价于 $[l_i, r_i]$ 和 $[2n - r_i + 1, 2n - l_i + 1]$ 相同。

这样用昨天那个题的方法，弄出等价类之后，还有 $O(n)$ 个“不是等价类”的限制。然后就，贪心的搜即可。

不过我们不需要这么麻烦：类似 Manacher，没有逃脱之前束缚的限制不用管，总合并次数为 $O(n)$ 。

回文 Border

如果 S 是一个回文串, T 是 S 的一个回文后缀, 容易发现 T 是 S 的 Border; 反之, 如果 T 是 S 的 Border, 则 T 是回文串。

搬出昨天讲的 Border 理论, 我们可以考察右端点相同的回文串具有的性质 (因为他们互为 Border): 可以划分为 $O(\log n)$ 个等差数列, 且每个等差数列对应着周期。

该性质可以优化很多与回文串相关的问题, 比如回文划分 (需要先学习回文自动机, 出于时间原因以及它确实不常用, 我们只给出[参考资料](#))。Manacher 结合线段树二分可以平替一部分 PAM。

回文划分问题: [P14771 \[ICPC 2024 Seoul R\] Palindromic Length](#), [CF932G Palindrome Partition](#).

特别的, CF932G 涉及到了一种“折叠字符串把 Border 变为回文串”的方法, 可以做[QOJ15877 回响形态](#)。

后缀结构的几个小问题

问题 1: 给定字符串 S , 定义 $T_i = S[i \cdots n]$, 将 T_i 按照字典序排序。

问题 2: 考虑将 T_i 插入一个字典树, 分析它的虚树 (也就是, 压缩 Trie, 一般称为后缀树) 结构 (比如, 本质不同子串个数)。

问题 3: 给定一棵树 T , 以及 n 个节点 $v_1, v_2, \cdots, v_n$, 满足 v 的 DFS 序是递增的。如果知道了相邻两点的 LCA 的深度, 如何求出任意两点 LCA 的深度?

后缀排序

这里介绍最基础的 $O(n \log n)$ 倍增 + 基数排序求 SA 的方法。

第一步，尝试倍增：对每个 k ，定义

$T_i = S[i \cdots \min(i + 2^k - 1, |S|)]$ ，维护 T_i 的字典序的相对大小关系。

第二步，倍增之后，视作 $(rank_i, rank_{i+2^{k-1}})$ 的二元组，求二元组的排序（这种技巧很常用）。由于值域都是 $O(n)$ ，容易 $O(n)$ 进行基数排序。

后缀排序的代码是很值得研究的问题，建议参考[OI-Wiki 上的实现](#)。

当然，如果你在考场上，可以先写二分 + 哈希，然后改成单 $\log$ 。后缀排序可以线性，而且并不是那种大常数纯鸡肋算法（是有显著优化的）。

Height 数组

定义 rk_i 为 $S[i \cdots n]$ 的排名, sa_i 表示排名为 i 的后缀是哪一个。

定义 $height_i = |\text{LCP}(S[sa_i \cdots n], S[sa_{i-1} \cdots n])|$, 特别的 $height_1 = 0$ 。

那么有结论: $height_{rk_{i+1}} \geq height_{rk_i} - 1$ 。因此, $height$ 数组可以 $O(n)$ 计算 (均摊)。

容易发现, $height$ 数组描述的就是后缀字典树上相邻两个节点 LCA 的深度。

本质不同的字符串树, 也就是后缀字典树的节点数, 为 $\frac{n(n+1)}{2} - \sum_{i=2}^n height_i$ 。

Height 数组

我们希望计算 $S[i \cdots n]$ 和 $S[j \cdots n]$ 的 LCP。不妨设 $rnk_i < rnk_j$ 。

根据问题 3，我们知道：答案就是 $\min_{rnk_i < k \leq rnk_j} height_k$ 。

因此，我们把后缀 LCP 问题，转化为了 RMQ 问题。我们可以：

- (1) 套用数据结构问题的常用方法，比如单调栈、ST 表；
- (2) 建立广义笛卡尔树（或者笛卡尔树，Kruskal 重构树等），容易发现本质上就是后缀树。

出于时间原因，Height 数组结合数据结构只提供[参考资料](#)（[参考其中 Height 数组运用的章节](#)）。

P3181 [HAOI2016] 找相同字符

题目大意

给定字符串 S 和 T ，求从 S 和 T 分别选出一个子串相同的方法数。

$$1 \leq |S|, |T| \leq 2 \times 10^5。$$

P3181 [HAOI2016] 找相同字符

考虑令 $A = S + c + T$ ，其中 c 是一个特殊字符。

$n = |S|$ ， $m = |T|$ ，则答案就是： $\sum_{i=1}^n \sum_{j=n+2}^{n+m+1} \text{LCP}(i, j)$ 。

这个问题就相当容易了：一种方法是，类似 Kruskal，按照 Height 从大到小合并集合；合并的时候计算一下贡献。

单调栈的方法其实和他本质相同。按照 Height 从大到小合并是一个很常见的技巧。

P14591 [LNCPC 2025] Kanon

题目大意

给定长度为 n 的字符串 S 。

对每个 $1 \leq k \leq n$ ，求将 S 划分为 k 个子串后按位异或的最大值，以及取到最大值的方案数，对 998244353 取模。

$|S| \leq 3 \times 10^6$ ，6s。

P14591 [LNCPC 2025] Kanon

判掉 $k = n$ 。容易想到，我们要最大化答案的二进制位数（不含前导 0）。

最长字符串长度至多为 $n - k + 1$ ，我们分类讨论：

- (1) 如果第一个 1 所在后缀长度 $\leq n - k$ ，那么显然所有分割都在整个前导 0 处；
- (2) 否则，我们将某个 1 和其后 $n - k$ 个字符分在一块，其他所有字符单独一段。

抛开末尾字符不谈，我们需要前 $n - k$ 个字符的字典序最大；而如果前 $n - k$ 个字符确定了，由于 1 的总奇偶性不变，因此最后一个字符也是固定的。

从大往小合并 $\text{height} \geq$ 某个值的相邻数，就可以做到 $O(n\alpha(n) + n \log n)$ ，瓶颈在于求 SA。

P7361 「JZOI-1」拜神

题目大意

定义 $f(T)$ 为最大的 L ，使得 T 中某个长度为 L 的子串出现了至少两次。

给定字符串 S 。 q 次询问，每次给出 $[l, r]$ ，计算 $f(S[l \cdots r])$ 。
 $1 \leq S \leq 5 \times 10^4$ ， $1 \leq q \leq 10^5$ ，2.5s。

P7361 「JZOI-1」拜神

显然答案具有可二分性，考虑如何判定 $f(S[l \cdots r]) \geq k$ 。那么， $\max_{l \leq i < j \leq r-k+1} \text{LCP}(i, j) \geq k$ 。

将所有 $\text{Height} \geq k$ 的元素合并到同一个连通块里，维护 next_i 表示连通块里 i 的后继。

那么等价于： $\min_{l \leq i \leq r-k+1} \text{next}_i \leq r - k + 1$ 。如果能对每个 k 维护 next_i （可持久化线段树），那么本题就可以在 $O(q \log^2 n)$ 的复杂度内解决询问部分。

在 Height 合并的过程中，使用启发式合并即可动态修改 next ，只会进行 $O(n \log n)$ 次修改。

故时间复杂度为 $O((n + q) \log^2 n)$ ，空间复杂度为 $O(n \log^2 n)$ 。

CF1110H Modest Substrings

题目大意

给定十进制数 l 和 r (没有签到 0)。

你需要求出一个长度为 n 的十进制数 A (可以有前导零), 使得 A 的合法子串数最多。并且最小化 A 的字典序。

一个字符串合法, 当且仅当它没有前导零, 且十进制下的数值在 l 和 r 之间。

$1 \leq l, r \leq 10^{800}$, $1 \leq n \leq 2000$, 5s。

CF1110H Modest Substrings

如果维护右端点相同的所有子串，显然不太处理好处理前导零，以及和 l 与 r 的关系。

但是维护左端点相同的子串是容易的：如果没有前导零，所有长度介于 $|l| + 1$ 和 $|r + 1| - 1$ 之间的子串都是合法的；而我们也只需要维护长度为 $|l|$ 的子串和 l 的相对大小关系，以及长度为 $|r + 1|$ 的子串和 $r + 1$ 的相对大小关系。

所以从右往左做。由于你可能要不断往前加字符往后删字符，因此你需要知道当前后缀和 l 、 $r + 1$ 每个后缀的相对大小关系。

构造 $S = l + "" + (r + 1) + ""$ ，其中 $""$ 是一个空字符（字典序最小），我们求 S 的后缀数组。

维护当前后缀在 S 所有后缀中的前继。如果 l 所代表的后缀在当前后缀之前，那么长度为 $|l|$ 的子串 $\geq l$ ；如果 $r + 1$ 所代表的后缀在当前后缀之前，那么长度为 $|r + 1|$ 的子串 $\geq r + 1$ 。

维护一个类似自动机的结构即可，复杂度 $O(|\Sigma|V^2 + nV|\Sigma|)$ 。

CF1043G Speckled Band

题目大意

给定字符串 S 。有 q 次询问，每次给出区间 $[l, r]$ ，求最小的整数 D ，使得可以将 $S[l \cdots r]$ 划分为若干个子串，且有一个子串出现了至少两次，本质不同的子串恰好有 D 种。

$1 \leq |S|, q \leq 2 \times 10^5, 7s。$

CF1043G Speckled Band

如果所有字符都出现了至多一次，直接输出 -1。

那么，我们一定可以取某个出现了两次的字符，这样 $D = 4$ ，我们只需要判断 D 是否能等于 1, 2, 3。

D 能否等于 1：即 $S[l \cdots r]$ 是否存在整周期，可以结合哈希 $O(\log n)$ 判断（因为只需要考虑素因子）。

D 能否等于 2：如果 $S[l \cdots r]$ 存在 Border，由 Border 的性质存在一个不重叠的 Border；否则，设第一个字符串为 A ，第二个字符串为 B 。如果头和尾同时出现 AB 那么就有 Border 矛盾了；否则，头为 AA 或者尾为 BB ，可以用《优秀的拆分》的方法处理。

D 能否等于 3：考虑第一个子串 A 。如果它出现了两次，就可以分为 $ABAC$ 这种形态——只需要 S_l 出现了至少两次； S_r 同理。如果这两个都不满足，只能形如 $ABBC$ ，还是《优秀的拆分》。

CF1043G Speckled Band

怎么判断一个子串是否存在 Border? 基本子串字典是可以的, 更简单的方法是:

我们取最小 Border, 长度为 k 。对 k 进行根号分治:

- (1) 如果 $k \leq \sqrt{n}$, 那么这样的 k 只有 $\sqrt{n}$ 个, 暴力枚举检验即可;
- (2) 如果 $k > \sqrt{n}$, 由最小性 $S[l \cdots l+k-1]$ 不存在非空 Border, 那么它在 S 中所有出现位置不重叠, 最多 $O(\sqrt{n})$ 次。因此这样的起始位置一定是后缀数组中 l 的某个不超过 $2\sqrt{n}$ 的邻域, 暴力枚举即可。

复杂度 $O(n\sqrt{n} + n \log n)$ 。

其他字符串问题

哪些东西被我们漏掉了呢？

- (1) 子序列自动机。这一块题目不多，比如 [P14775 \[ICPC 2024 Seoul R\] String Rank](#)；
- (2) Lyndon 分解，我不会，而且可以认为这个东西一点用都没有。
- (3) SAM，我几乎不会。本质上就几个概念：endpos 集合，parent tree；理顺了代码很好写，但是复杂度证明我也不会，可以阅读[这篇文章](#)做基础的了解；
- (4) 广义 SAM，可以参考[这篇文章](#)和[这篇文章](#)；
- (5) 基本子串字典，参考《Border 的四种求法》和[这篇文章](#)；
- (6) [最长公共子序列科技 1](#)，[最长公共子序列科技 2](#)。

以上的内容，除非你特别闲，否则省选前可以不接触，学他们的价值远没有去练数据结构大。